TP N.º 8 · Food Store: la base de la tablet y el cierre de la capa de datos

Base de Datos II · UTN, Tecnicatura Universitaria en Programación · Grupo 10: Mariano Chirino, Andrés Fabre y Facundo Quiroga · Repositorio `github.com/marianoemir/TP-Unidad8-BD2` (etiqueta `tp8`)

Contiene 1.a, 2.c, 3.a, 3.b y 3.c. Cada valor indica el archivo de `evidencias/` que lo respalda. Todo se probó en una copia de la base con datos inventados (PostgreSQL 17.10 y SQLite 3.54.1, Windows). **Regla de oro:** el móvil propone, el servidor dispone; `SIMULADO_APROBADO` no es una venta.

1.a · Qué datos viajan a la tablet

La tablet recibe una instantánea del catálogo con la vista `v_snapshot_movil` (PostgreSQL → CSV → `snapshot_servidor` → `producto_cache`). La vista tiene 7 columnas y ninguna es de usuarios (`evidencias/13`).

Dato en PostgreSQL	¿Viaja?	En la tablet	Por qué
`producto.id`, `producto.nombre`	Sí	`producto_id`, `nombre`	Une el borrador con el producto del servidor; el vendedor lo ve en pantalla
`producto.precio`	Sí, convertido	`precio_centavos`	Entero en centavos: cuentas exactas; se convierte una sola vez en PostgreSQL
`producto.stock`	Sí, orientativo	`stock_visto`	Solo se muestra; la decisión de stock es del servidor
`producto.disponible`	Sí	`disponible_visto`	Ofrecer solo lo que se puede vender
`producto.eliminado` (o su categoría)	Sí, como lápida	`eliminado`	La baja viaja como dato: sale de pantalla pero la fila se conserva, porque un borrador puede usarla
Fecha de la instantánea	Sí	`capturado_en`	Se sabe qué tan vieja es la lista
`descripcion`, `imagen`, `categoria_id`, `created_at`	No	—	No hacen falta para vender; la categoría solo sirve en el servidor para calcular la lápida
`usuario.*` (nombre, mail, celular, contraseña, rol)	No	—	Riesgo si la tablet se pierde; al borrador solo viaja `usuario_id_servidor`
`pedido` y `detalle_pedido` confirmados	No	—	Solo el servidor confirma y numera las ventas

Por qué y quién manda. A la tablet viaja solo lo necesario para mostrar el catálogo y armar borradores sin señal: guardar poco es la principal medida de seguridad cuando SQLite no tiene usuarios ni permisos. La fuente de verdad es **PostgreSQL**, que decide precio, stock y ventas.

2.c · Los casos C2 a C6

Preparación común: se anotan los borradores con la lista vigente, el servidor cambia datos de la copia de pruebas, se exporta una instantánea nueva (`11_exportar_snapshot.sql`), se importa `snapshot_servidor`, se recarga `producto_cache` y se corre `movil/semaforo.sql`. Instantáneas: `2026-11-03 08:48:12-03` (2.a, 16 productos, `evidencias/03`) y `2026-11-03 08:50:45-03` (después de los cambios, `evidencias/07`).

Caso	Qué se preparó (comandos)	Resultado	Explicación
C2 precio cambió	`casos_c2_c5_borradores.sql` (1 Muzzarella a 350000); `psql -f postgresql/casos/c2_subir_precio.sql` (sube a 400000); semáforo; `casos_c2_c6_resolucion.sql` (el cliente acepta, solicitud nueva); semáforo	1.ª corrida: `CONFLICTO`, «precio de Muzzarella cambió de 350000 a 400000 centavos». 2.ª: solicitud nueva `SIMULADO_APROBADO` (`evidencias/08` y `10`)	El precio del borrador ya no coincide pero no falta nada: amarillo, se le pregunta al cliente. Al aceptar se actualiza el borrador y se encola un UUID nuevo, que el índice parcial `ux_cola_una_pendiente` permite porque la anterior ya no está `PENDIENTE`
C3 sin stock	Borrador de 10 Tiramisú; semáforo (hay 6)	`RECHAZADO`: «stock insuficiente de Tiramisú (pide 10, hay 6)» (`evidencias/08`)	El `stock_visto` de la tablet es orientativo: decide el servidor, y el semáforo compara con el stock de la instantánea. El stock de PostgreSQL no se tocó
C4 producto dado de baja	`psql -f postgresql/casos/c4_baja_logica.sql` (Fugazzeta `eliminado = TRUE`); `cargar_catalogo.sql`; semáforo	`RECHAZADO`: «producto 3 (Fugazzeta) dado de baja». En `v_catalogo_disponible`: 0 filas; en `producto_cache` la fila sigue con `eliminado = 1` (`evidencias/07` y `08`)	La baja viaja como lápida. La carga usa `INSERT … ON CONFLICT DO UPDATE`, no `INSERT OR REPLACE`, para conservar la fila que un borrador referencia con clave foránea
C5 todo coincide	Borrador sin cambios; semáforo	`SIMULADO_APROBADO`, sin motivo (`evidencias/08`)	Pasaría las validaciones, pero **no es una venta**: solo `sp_crear_pedido` en el servidor la confirma, y revisa el stock otra vez con `SELECT … FOR UPDATE`
C6 mismo UUID	Se intenta insertar de nuevo `c5000000-…-0b` y se repite el semáforo	`UNIQUE constraint failed: cola_sync.solicitud_uuid`; la fila original conserva su estado y su `evaluada_en`; 3.ª corrida: `solicitudes_evaluadas = 0` (`evidencias/09` y `11`)	El UUID es la clave primaria, así que el duplicado no entra; y como el semáforo solo evalúa `PENDIENTE`, repetirlo no reevalúa nada

Dónde iría la protección real (parada 8). En el servidor, no solo en la tablet: una tabla `solicitud_movil` con el UUID como clave primaria, insertada con `ON CONFLICT DO NOTHING` en la misma transacción que crea el pedido con `sp_crear_pedido`. Si el servidor no recordara el UUID, un reintento crearía un segundo pedido. (Ejemplo docente: el TP no lo implementa.)

PostgreSQL no cambió. `postgresql/verificar_no_cambio.sql` (solo lectura), antes (`evidencias/01`) y después (`evidencias/12`): **6 pedidos, 13 detalles, stock total 454, huella** `89f8351d0162e23ef1473f32b05ce221`, idénticos. El precio de la Muzzarella y la baja de la Fugazzeta los cambió el servidor de pruebas a propósito, para preparar C2 y C4; no alteran pedidos ni stock. La tablet nunca escribió en PostgreSQL.

Casos borde del semáforo (revisión cruzada). `movil/casos_f2_semaforo.sql` sobre una copia de la tablet (`evidencias/18` y `19`): rojo y amarillo en el mismo borrador da `RECHAZADO` (gana el rojo); un producto fuera de la instantánea, un borrador sin ítems y un producto no disponible dan `RECHAZADO`; el control da `SIMULADO_APROBADO`; una segunda solicitud `PENDIENTE` del mismo borrador falla con `UNIQUE constraint failed: cola_sync.id_local`.

3.a · Mapa de la capa de datos

PostgreSQL es la única fuente de verdad: ninguno de los otros dos motores decide precio, stock ni ventas.

Motor	Dónde vive	Qué guarda	Quién manda
PostgreSQL	Servidor (`foodstore`; `foodstore_u8` en las pruebas)	Catálogo (`producto`, `categoria`), usuarios, `pedido`, `detalle_pedido`, stock; publica `v_snapshot_movil`	**Fuente de verdad.** Decide precio, stock y ventas; solo `sp_crear_pedido` confirma una venta y revalida el stock
MongoDB (Unidad 7)	Base `foodstore_nosql`	Colección `pedidos`: un documento por pedido, con cliente y líneas embebidos. 7 pedidos migrados (1, 2, 3, 5 y 6 de `data.sql`, más el 7 y el 8 creados con `sp_crear_pedido` para la Unidad 7); el 4 está dado de baja	Solo sobre esos documentos: es una copia migrada para lectura de pedidos completos; no decide stock, precio ni ventas
SQLite (Unidad 8)	`movil/foodstore_movil.db`, en la tablet	`producto_cache` (copia con fecha), `pedido_borrador`, `item_borrador` y `cola_sync` (cola y resultado)	**Nadie**: solo consulta y prepara. El semáforo simula; `SIMULADO_APROBADO` no es una venta

Dato	Manda	Dónde se copia
Precio, stock, disponibilidad y bajas del catálogo	PostgreSQL	SQLite (copia con fecha)
Ventas y numeración de pedidos	PostgreSQL	MongoDB (copia de pedidos ya existentes)
Borradores y estado de su envío	La tablet los prepara; PostgreSQL decidiría la venta real	Solo en SQLite
Datos de usuarios	PostgreSQL	No viajan a la tablet

Flujo. PostgreSQL publica la instantánea → la tablet la guarda y arma borradores sin señal → al volver la señal, cada solicitud se compara con una instantánea nueva (en este TP es una simulación y no escribe en PostgreSQL) → solo PostgreSQL confirmaría la venta. MongoDB no participa de la sincronización ni toca stock o ventas.

3.b · Plan de mantenimiento (agregado al plan de la Unidad 6)

Se apoya en `SPEC_U6.md` y en el simulacro de la Unidad 6 (informe del 21/10/2026); acá solo lo que cambia al sumar las tablets.

¿Qué se respalda y cómo se prueba la restauración? La fuente de verdad: la base PostgreSQL con `pg_dump` en formato custom (`backup_foodstore.ps1`), con fecha y hora en el nombre, verificado con `pg_restore --list` y guardado fuera del repo. Según la spec de la Unidad 6 el respaldo es **diario** (pérdida máxima prevista de 24 h) y se saca otro antes de cada cambio estructural; al 21/10/2026 se generaba a mano, sin tarea programada. La restauración se prueba **cada semana** en una base descartable aparte, nunca sobre la real, con `restore_test.ps1` (conteos y controles lógicos); en el simulacro del 18/10/2026 tardó 6,19 s. Se suma `verificar_no_cambio.sql`: si pedidos, detalles, stock total y huella coinciden con los de la base original, el respaldo sirve. Limitación conocida: `pg_dump` no incluye roles ni permisos (mejora propuesta, no implementada). MongoDB se regenera desde PostgreSQL con la exportación de la Unidad 7. La base de la tablet no se respalda como fuente (la lista de precios se regenera con una instantánea nueva), pero se copia `foodstore_movil.db` antes de actualizar el catálogo, porque los borradores no enviados solo existen ahí.

Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets? Quedan con datos «del futuro»: precios, stock y bajas que el servidor restaurado ya no tiene. **No envían nada con esa lista**: primero vuelven a descargar el catálogo (instantánea nueva y recarga de `producto_cache`) y recién entonces se corre el semáforo sobre las solicitudes `PENDIENTE`. Como el UUID no cambia, reenviarlas no duplica pedidos, y como `SIMULADO_APROBADO` nunca fue una venta, la restauración no deja ventas falsas en las tablets.

¿Cada cuánto y quién revisa los `CONFLICTO` y `RECHAZADO`? Al volver la señal y al cierre de cada jornada de feria, el **vendedor a cargo de la tablet junto con el administrador (rol `ADMIN`)**. Ante un `CONFLICTO` se le pregunta al cliente si acepta el precio nuevo y, si acepta, se actualiza el borrador y se genera una solicitud nueva; ante un `RECHAZADO` se corrige el borrador o se avisa que el producto no está disponible, sin stock o dado de baja. Nada se resuelve desde la tablet contra el servidor.

3.c · Code review con IA

Revisión hecha por Facundo con **OpenCode en el agente Plan** (solo lectura), en la rama `review-final`. No consta el modelo, el prompt ni la fecha de la corrida: no se registraron. Ningún hallazgo se dio por cierto sin comprobarlo: las pruebas las repitió Mariano en una copia de las bases (`evidencias/15` a `17`; script `postgresql/verificar_hallazgos_3c.sql`). Decisiones: aceptado, corregido o descartado.

ID	Dónde (unidad)	Qué dijo la IA	Cómo se comprobó en la base	Decisión y motivo
H36	`movil/pruebas_sqlite.sql` (U8)	El CLI de SQLite no hace `ROLLBACK` solo ante un error, así que un `COMMIT` posterior dejaría un borrador a medias	Se corrió el script en una base limpia: los 3 errores de C7 ocurren y los conteos quedan en 1 borrador, 2 ítems y 1 solicitud antes y después (`evidencias/17`)	**Descartado.** Lo general del CLI es cierto, pero este script no lo sufre: cada bloque de C7 termina en `ROLLBACK` y no hay ningún `COMMIT` después de un error
H01	`10_snapshot_catalogo.sql` y `sp_crear_pedido` (base heredada)	La vista marca como baja los productos de una categoría eliminada, pero `sp_crear_pedido` solo mira `producto.eliminado`	En una copia: categoría 5 dada de baja; la vista da 16 filas con 4 lápidas y `sp_crear_pedido` **aceptó** un pedido del producto 12 (stock 24 a 23) (`evidencias/16`)	**Aceptado.** La asimetría es real: la tablet rechazaría el pedido y el servidor lo deja pasar. No se toca el procedimiento heredado; se documenta
H04	`detalle_pedido` (base heredada)	Falta un índice por `producto_id`; el único útil es el compuesto `UNIQUE (pedido_id, producto_id)`	`EXPLAIN`: `Seq Scan` (costo 20.50) con 13 filas; con `enable_seqscan = off`, `Bitmap Index Scan` sobre el compuesto (costo 22.91) (`evidencias/15`)	**Descartado con fundamento.** Con 13 filas un índice nuevo no mejora nada y `conventions.md` indica «no se agregan índices en este TP». Con volumen real convendría uno que empiece por `producto_id`
R1	`sp_crear_pedido` (base heredada)	Un `producto_id` repetido en `p_items` rompe sin mensaje de negocio	Se llamó con el producto 4 dos veces: `SQLSTATE 23505` sobre `detalle_pedido_pedido_id_producto_id_key`; la transacción se revierte entera (`evidencias/16`)	**Aceptado.** El error es genérico pero no deja nada a medias. Se documenta; no se modifica el código heredado

Resumen: 4 hallazgos de la Unidad 8 y de la base heredada (Unidades 1 a 6); 2 descartados con fundamento (H36 porque la ejecución lo desmiente, H04 por volumen y convención) y 2 aceptados y comprobados; ninguno corregido, porque no se modificó código heredado. En la bitácora se declara qué propuso la IA y qué se comprobó aparte; no consta cuántos hallazgos devolvió OpenCode en total ni cuáles de H01 y R1 salieron de ella.